

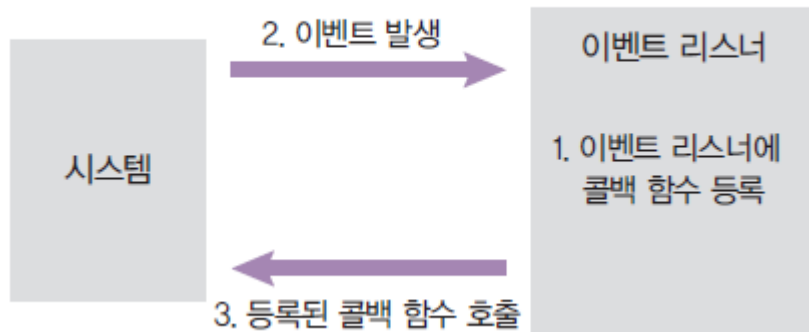
노드

- Node.js®는 크롬 V8 자바스크립트 엔진으로 빌드된 자바스크립트 런타임입니다.

노드의 특성 1: 이벤트 기반

- 이벤트가 발생할 때 미리 지정해둔 작업을 수행하는 방식
 - 이벤트의 예: 클릭, 네트워크 요청, 타이머 등
 - 이벤트 리스너: 이벤트를 등록하는 함수
 - 콜백 함수: 이벤트가 발생했을 때 실행될 함수

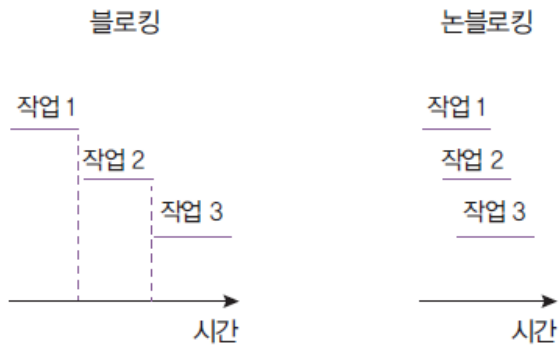
▼ 그림 1-4 이벤트 기반



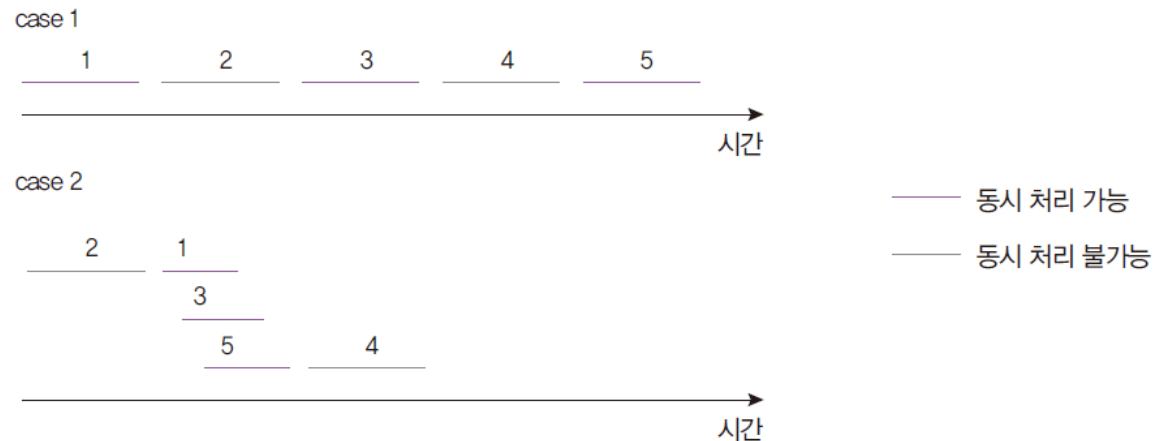
노드의 특성 2: 논블로킹 I/O

- 오래 걸리는 함수를 백그라운드로 보내서 다음 코드가 먼저 실행되게 하고, 나중에 오래 걸리는 함수를 실행
 - 논 블로킹 방식 하에서 일부 코드는 백그라운드에서 병렬로 실행됨
 - 일부 코드: I/O 작업(파일 시스템 접근, 네트워크 요청), 압축, 암호화 등
 - 나머지 코드는 블로킹 방식으로 실행됨
 - I/O 작업이 많을 때 노드 활용성이 극대화

▼ 그림 1-9 블로킹과 논블로킹



▼ 그림 1-10 동시 처리로 얻는 시간적 이득



논블로킹 vs 비동기

- 논블로킹: 제어권을 잃지 않음
- 비동기: 작업 완료를 기다리지 않음
- 블로킹 동기 모델
 - 제어권을 넘기고 작업 완료를 기다림
- 블로킹 비동기 모델
 - 작업 완료를 기다리지 않으나 제어권을 넘겼기 때문에 다음 코드를 실행할 수 없어 의미가 없음
- 논블로킹 동기 모델
 - 제어권을 가지고 있으나 작업 완료를 기다려야 하기 때문에 작업이 완료되었는지 직접 확인해야 함
- 논블로킹 비동기 모델
 - 제어권을 가지고 있고 작업 완료도 기다리지 않기 때문에 바로 다음 코드를 실행

프로세스 vs 스레드

- 프로세스와 스레드
 - 프로세스: 운영체제에서 할당하는 작업의 단위, 프로세스 간 자원 공유X
 - 스레드: 프로세스 내에서 실행되는 작업의 단위, 부모 프로세스 자원 공유
- 노드 프로세스는 멀티 스레드이지만 직접 다룰 수 있는 스레드는 하나이기 때문에 싱글 스레드라고 표현

▼ 그림 1-13 스레드와 프로세스

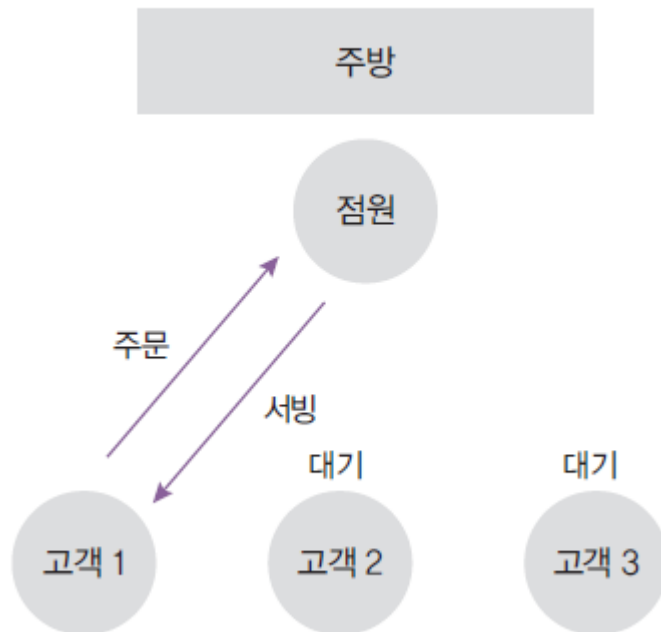
- 노드는 주로 멀티 스레드 대신 멀티 프로세스 활용
- 노드는 14버전부터 멀티 스레드 사용 가능



싱글 스레드

- 싱글 스레드라 주어진 일을 하나밖에 처리하지 못함
 - 블로킹이 발생하는 경우 나머지 작업은 모두 대기해야 함 -> 비효율 발생
- 주방에 비유(점원: 스레드, 주문: 요청, 서빙: 응답)

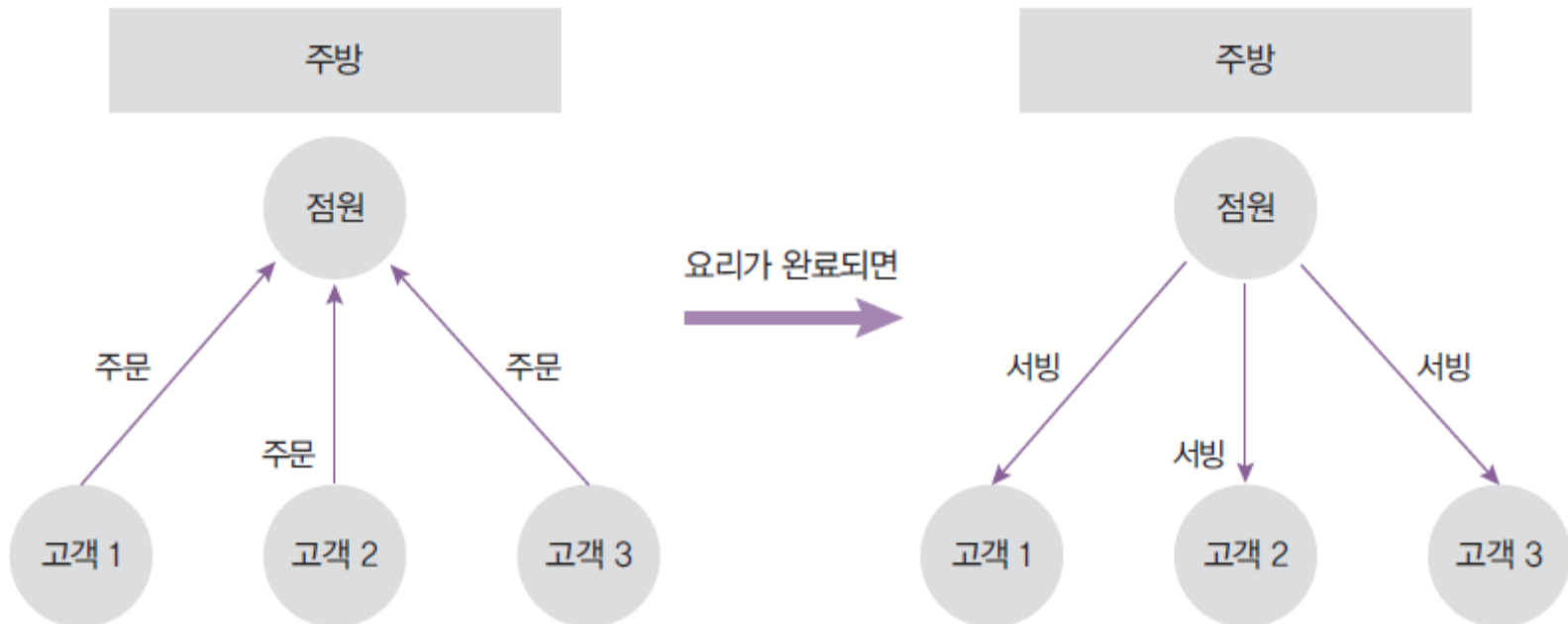
▼ 그림 1-10 싱글 스레드, 블로킹 모델



싱글 스레드

- 대신 논 블로킹 모델을 채택하여 일부 코드(I/O)를 백그라운드(다른 프로세스)에서 실행 가능
 - 요청을 먼저 받고, 완료될 때 응답함
 - I/O 관련 코드가 아닌 경우 싱글 스레드, 블로킹 모델과 같아짐

▼ 그림 1-11 싱글 스레드, 논블로킹 모델



멀티 스레드 모델과의 비교

- 싱글 스레드 모델은 에러를 처리하지 못하는 경우 멈춤
 - 프로그래밍 난이도 쉽고, CPU, 메모리 자원 적게 사용
- 멀티 스레드 모델은 에러 발생 시 새로운 스레드를 생성하여 극복
 - 단, 새로운 스레드 생성이나 놓고 있는 스레드 처리에 비용 발생
 - 프로그래밍 난이도 어려움
 - 스레드 수만큼 자원을 많이 사용함
- 점원: 스레드, 주문: 요청, 서빙: 응답

▼ 그림 1-12 멀티 스레드, 블로킹 모델



멀티 스레드의 활용

■ 노드 14버전

- 멀티 스레드를 사용할 수 있도록 worker_threads 모듈 도입
- CPU를 많이 사용하는 작업인 경우에 활용 가능
- 멀티 프로세싱만 가능했던 아쉬움을 달래줌.

▼ 표 1-1 멀티 스레딩과 멀티 프로세싱 비교

멀티 스레딩	멀티 프로세싱
하나의 프로세스 안에서 여러 개의 스레드 사용	여러 개의 프로세스 사용
CPU 작업이 많을 때 사용	I/O 요청이 많을 때 사용
프로그래밍이 어려움	프로그래밍이 비교적 쉬움

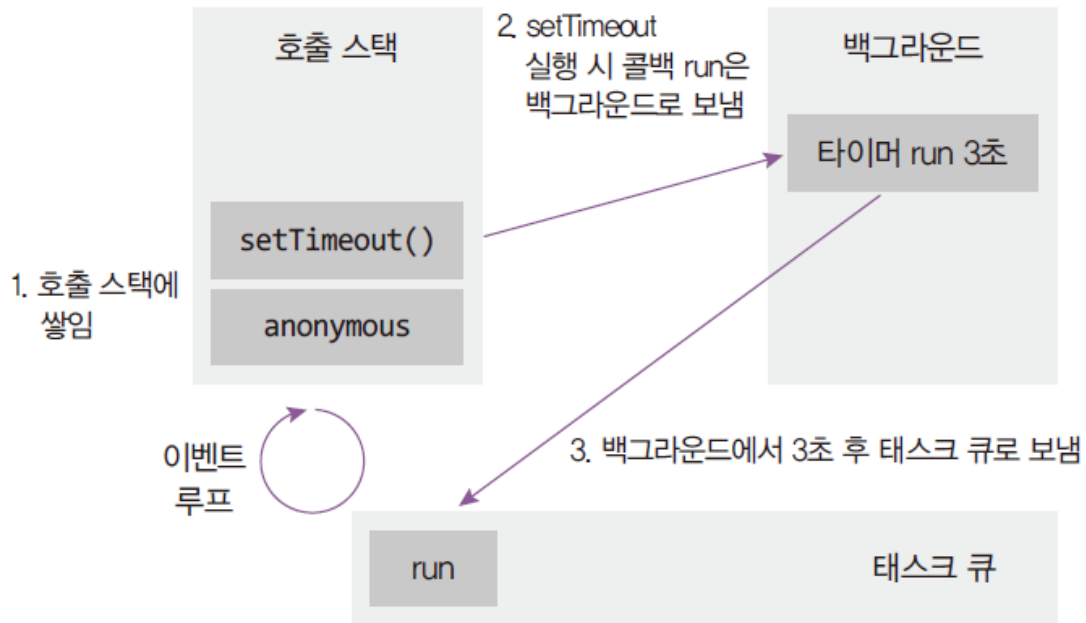
노드 서버의 장단점

▼ 표 1-1 노드의 장단점

장점	단점
멀티 스레드 방식에 비해 컴퓨터 자원을 적게 사용함	싱글 스레드라서 CPU 코어를 하나만 사용함
I/O 작업이 많은 서버로 적합	CPU 작업이 많은 서버로는 부적합
멀티 스레드 방식보다 쉬움	하나뿐인 스레드가 멈추지 않도록 관리해야 함
웹 서버가 내장되어 있음	서버 규모가 커졌을 때 서버를 관리하기 어려움
자바스크립트를 사용함	어중간한 성능
JSON 형식과 호환하기 쉬움	

이벤트 루프

▼ 그림 1-6 이벤트 루프 1



■ 이벤트루프 구조

- 이벤트 루프: 이벤트 발생(`setTimeout` 등) 시 호출할 콜백 함수들(위의 예제에서는 `run`)을 관리하고, 호출할 순서를 결정하는 역할
- 태스크 큐: 이벤트 발생 후 호출되어야 할 콜백 함수들이 순서대로 기다리는 공간
- 백그라운드: 타이머나 I/O 작업 콜백, 이벤트 리스너들이 대기하는 공간. 여러 작업이 동시에 실행될 수 있음

이벤트 루프

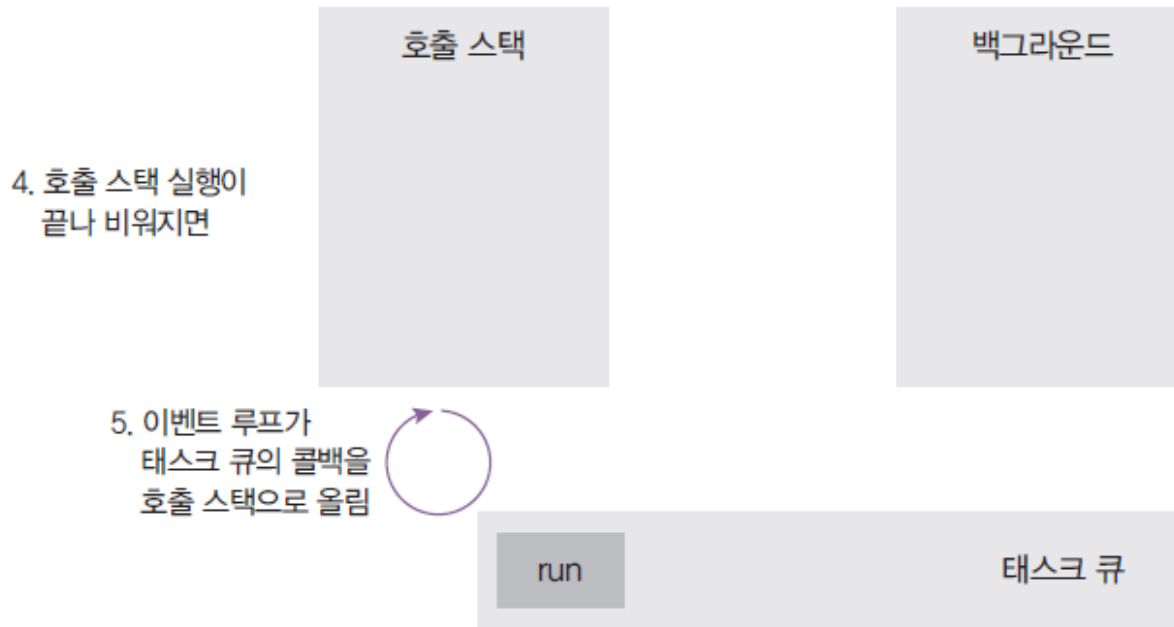
▼ 그림 1-7 이벤트 루프 2



- 예제 코드에서 `setTimeout`이 호출될 때 콜백 함수 `run`은 백그라운드로
 - 백그라운드에서 3초를 보냄
 - 3초가 다 지난 후 백그라운드에서 태스크 큐로 보내짐

이벤트 루프

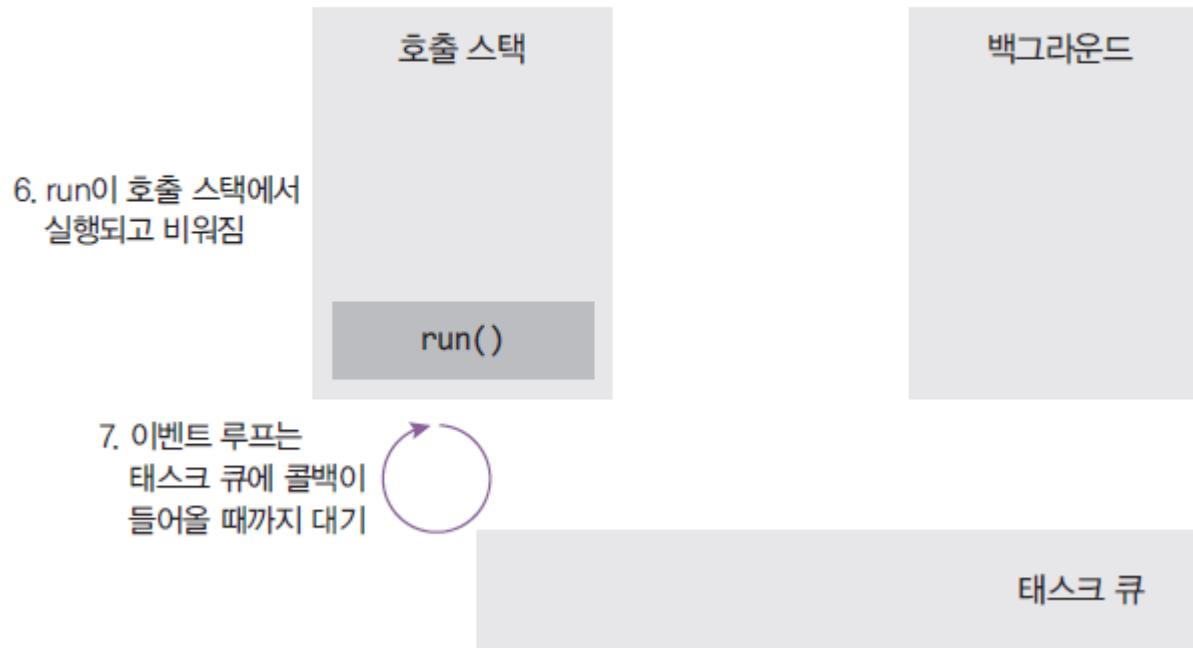
▼ 그림 1-7 이벤트 루프 2



- setTimeout과 anonymous가 실행 완료된 후 호출 스택이 완전히 비워지면, 이벤트 루프가 태스크 큐의 콜백을 호출 스택으로 올림
 - 호출 스택이 비워져야만 올림
 - 호출 스택에 함수가 많이 차 있으면 그것들을 처리하느라 3초가 지난 후에도 run 함수가 태스크 큐에서 대기하게 됨 -> 타이머가 정확하지 않을 수 있는 이유

이벤트 루프

▼ 그림 1-8 이벤트 루프 3



- run이 호출 스택에서 실행되고, 완료 후 호출 스택에서 나감
 - 이벤트 루프는 태스크 큐에 다음 함수가 들어올 때까지 계속 대기
 - 태스크 큐는 실제로 여러 개고, 태스크 큐들과 함수들 간의 순서를 이벤트 루프가 결정함

프로미스

- Promise는 비동기 작업의 완료 또는 실패를 나타내는 객체
 - 미래에 어떤 값이 준비될 것을 약속하고, 완료/실패 시 콜백을 등록할 수 있는 객체
 - Promise는 3가지 상태를 가짐

상태	의미
pending	아직 완료되지 않은 상태
fulfilled	성공적으로 완료되어 값이 있는 상태 (resolve 호출)
rejected	실패한 상태 (reject 호출)

- new Promise(executor) 형태로 생성
- executor 함수는 동기적으로 실행되며 두 개의 인자를 받음
 - resolve(value) → 작업 성공 시 호출
 - reject(reason) → 작업 실패 시 호출

```
let p = new Promise((resolve, reject) => {  
  // 비동기 작업 수행  
});
```

프로미스

- Resolve(성공 리턴 값) -> then으로 연결
- Reject(실패 리턴 값) -> catch로 연결
- Finally 부분은 무조건 실행됨

```
const promise = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    const success = true;  
    if (success) resolve("done");  
    else reject("error");  
  }, 1000);  
});  
  
promise  
  .then(value => console.log("Fulfilled:", value))  
  .catch(err => console.log("Rejected:", err));
```


프로미스

■ 프로미스의 then 연달아 사용 가능 (프로미스 체이닝)

- resolve()가 호출되면 promise가 fulfilled 상태가 됨
 - 그 즉시 .then(...) 체인에 등록된 microtask가 실행 큐에 들어감
- then 안에서 return한 값이 다음 then으로 넘어감
- return 값이 프로미스면 resolve 후 넘어감
- 에러가 난 경우 바로 catch로 이동
 - 에러는 catch에서 한 번에 처리

promise

```
.then(v => v + 1)  
.then(v => v * 2)
```

```
new Promise((resolve, reject) => {  
  resolve('a');  
}).then(msg => {  
  return new Promise((resolve, reject) => {  
    resolve(msg + 'b');  
  });  
}).then((msg) => {  
  return new Promise((resolve, reject) => {  
    resolve(msg + 'c');  
  });  
}).then((msg) => {  
  return new Promise((resolve, reject) => {  
    console.log(msg);  
  });  
}).catch(err => console.error(err));
```

프로미스

- `Promise.resolve(성공리턴값)`: 바로 `resolve`하는 프로미스
- `Promise.reject(실패리턴값)`: 바로 `reject`하는 프로미스

```
var promise1 = Promise.resolve('success1');  
var promise2 = Promise.resolve('success2');
```

```
Promise.all([promise1, promise2])  
  .then(msg => console.log(msg))  
  .catch(err => console.error(err));
```

- `Promise.all(배열)`: 여러 개의 프로미스를 동시에 실행
 - 하나라도 실패하면 `catch`로 감
 - `allSettled`로 실패한 것만 추려낼 수 있음

async / await

- 비동기 작업을 동기 코드처럼 작성할 수 있게 해주는 문법
- async
 - 함수 앞에 붙이면 해당 함수는 항상 Promise를 반환
- await
 - async 함수 안에서만 사용할 수 있고, Promise 결과를 기다리고 반환

```
function fetchData() {  
  return new Promise((resolve) => {  
    setTimeout(() => resolve("데이터 받음"), 1000);  
  });  
}
```

```
async function getData() {  
  console.log("데이터 요청 중...");  
  const data = await fetchData();  
  console.log(data); // 1초 후 "데이터 받음"  
}
```

```
getData();
```

async / await

- await로 처리한 비동기 작업도 try/catch로 쉽게 잡을 수 있음

```
async function fetchWithError() {  
  try {  
    const data = await Promise.reject("에러 발생");  
  } catch (err) {  
    console.error(err); // "에러 발생"  
  }  
}  
  
fetchWithError();
```

- 여러 await를 순차적으로 쓰면 느릴 수 있으며, 병렬 처리하려면 Promise.all 사용

```
async function fetchAll() {  
  const [a, b] = await Promise.all([fetchData(), fetchData()]);  
  console.log(a, b);  
}
```

for await of

- 노드 10부터 지원
- for await (변수 of 프로미스 배열)
 - resolve된 프로미스가 변수에 담겨 나옴
 - await을 사용하기 때문에 async 함수 안에서 해야함

```
var promise1 = Promise.resolve('success1');
var promise2 = Promise.resolve('success2');

const f = async () => {
  for await (msg of [promise1, promise2]) {
    console.log(msg);
  }
};

f();
```

타이머 메서드

- set 메서드에 clear 메서드가 대응됨
 - set 메서드의 리턴 값(아이디)을 clear 메서드에 넣어 취소
- **setTimeout(콜백 함수, 밀리초)**
 - 주어진 밀리초(1000분의 1초) 이후에 콜백 함수를 실행합니다.
- **setInterval(콜백 함수, 밀리초)**
 - 주어진 밀리초마다 콜백 함수를 반복 실행합니다.
- **setImmediate(콜백 함수)**
 - 콜백 함수를 즉시 실행합니다.
- **clearTimeout(아이디):** setTimeout을 취소합니다.
- **clearInterval(아이디):** setInterval을 취소합니다.
- **clearImmediate(아이디):** setImmediate를 취소합니다.

Event loop

이벤트 루프는 Node.js의 Main thread로, 내부의 각 Phase를 돌면서 앱을 실행

각 Phase는 Queue로 이루어져 있으며,
Queue의 모든 Job을 수행하거나, 제한 갯수까지 실행한 후에 다음 Phase로 이동

- 순환 실행

timers

pending callbacks

idle, prepare

poll

check

close callbacks

timers

Phase: timer

대상: setTimeout(func, delay), setInterval(func, delay)

처리 작업: delay가 지났으면, 등록된 callback 실행

Phase: poll

대상: I/O

처리 작업: 대부분의 callback 실행

Phase: check

대상: setImmediate(func)

처리 작업: 등록된 callback 실행

setTimeout vs setImmediate

1. Main 모듈에서 호출되면, 호출되는 시점이 다름

```
setTimeout(() => console.log('timeout'), 0);  
setImmediate(() => console.log('immediate'));
```

Timeout(func, 0)의 0ms는 1ms로 동작

- 1) timeout의 cb가 timers queue에 등록
- 2) immediate의 cb가 check queue에 등록
- 3) Loop 시작, timers phase
- 3.A) 1ms 전 이라면 check queue에 등록된 immediate의 cb가 먼저 실행
- 3.B) 1ms 후 라면 timers queue에 등록된 timeout의 cb가 먼저 실행

2. I/O cycle 에서는, immediate 콜백이 항상 먼저 실행

```
require('fs').readFile('timer.js', _ => {  
  setTimeout(() => console.log('timeout'), 0);  
  setImmediate(() => console.log('immediate'));  
});
```

- 1) fs.readFile()이 실행되고, cb가 poll queue에 등록
- 2) poll phase에 진입
- 3) timeout의 cb가 timers queue에 등록
- 4) immediate의 cb가 check queue에 등록
- 5) poll queue의 Job을 모두 소진했으니, 다음 phase인 check phase로 이동

process.nextTick

Event loop의 일부가 아니며

process.nextTick(cb)의 callback은 nextTick queue에 등록되며, 각 phase 사이 사이에서 호출됨

process.nextTick() is not technically part of the event loop.

Instead, the nextTickQueue will be processed after the current operation is completed, regardless of the current phase of the event loop.

```
setTimeout(() => console.log('timeout1'), 0);  
setImmediate(() => console.log('immediate1'));
```

```
process.nextTick(() => {  
  setTimeout(() => console.log('timeout2'), 0);  
  setImmediate(() => {  
    process.nextTick(() => console.log('tick2'));  
    console.log('immediate2');  
  });  
  
  console.log('tick1');  
});
```

- 1) timeout의 cb가 timers queue에 등록
- 2) immediate의 cb가 check queue에 등록
- 3) nextTick의 cb가 nextTick queue에 등록되고, 가장 먼저 실행
- 4) nextTick의 callback의 Timeout과 Immediate의 각각의 cb가 queue에 등록

nextTick, microtask, macrotask

- macrotask queue (or task queue)

- 이벤트 루프에서 처리되는 비동기 작업들의 큐

- 실행 시점: 이벤트 루프의 각 단계(stage)에서 하나씩 꺼내어 실행

- microtask queue

- 이벤트 루프에서 현재 실행 중인 동기 코드가 끝난 직후 처리되는 작업들의 큐

- 작은 비동기 작업을 "즉시" 처리하여, 다음 macrotask가 실행되기 전에 상태를 업데이트하기 위해 사용

- Promise.then
- Promise.catch
- Promise.finally
- queueMicrotask(): **microtask** 큐에 콜백을 넣는 함수

```
queueMicrotask(() => console.log("microtask"));
```

nextTick, microtask, macrotask

- nextTick queue

- 이벤트 루프의 현재 단계가 끝나기 전에 무조건 먼저 실행되는 작업들의 큐

큐	예시	우선 순위	실행 시점
macrotask	setTimeout, setInterval, I/O	낮음	이벤트 루프 단계 끝난 후
microtask	Promise.then, queueMicrotask	nextTick 다음	동기 코드 끝난 직후
nextTick	process.nextTick	가장 높음	동기 코드 끝난 직후

nextTick queue의 작업이 항상 먼저 실행됨

이벤트 비동기 실행 순서: nextTick -> promise -> timeout

```
const promise = new Promise(resolve => {  
  console.log("promise");  
  resolve();  
  
  setTimeout(() => console.log("timeout in promise"));  
  process.nextTick(() => console.log("nextTick in promise"));  
}).then(() => console.log("then"));  
  
process.nextTick(() => console.log("nextTick in global"));  
console.log("global");
```

중첩 setTimeout vs setInterval

중첩 `setTimeout`은 지연 간격을 보장하지만
`setInterval`은 지연 간격을 보장하지 않음

```
function func(i) {  
  console.log(i);  
}
```

```
let i = 0;  
setInterval(() => func(++i), 100);
```

`func` 호출 사이의 '소모되는' 시간도 지연 간격에 포함되기 때문에,
`func` 호출 사이의 지연 간격이 실제 명시한 간격(100ms)보다 짧음

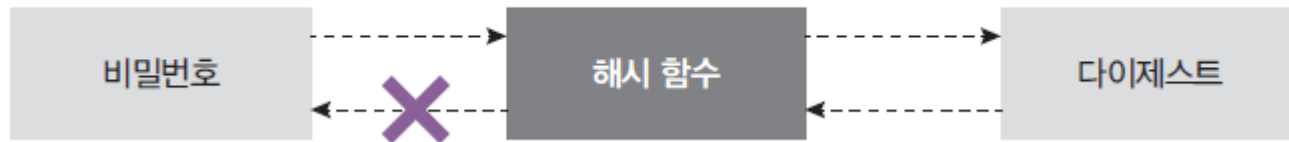
```
function func(i) {  
  console.log(i);  
}
```

```
setTimeout(function run() {  
  func(++i);  
  setTimeout(run, 100);  
, 100);
```

단방향 암호화(crypto)

- 암호화는 가능하지만 복호화는 불가능
 - 암호화: 평문을 암호로 만들
 - 복호화: 암호를 평문으로 해독
- 단방향 암호화의 대표 주자는 해시 기법
 - 문자열을 고정된 길이의 다른 문자열로 바꾸는 방식
 - abcdefgh 문자열 -> qvew

♥ 그림 3-8 해시 함수



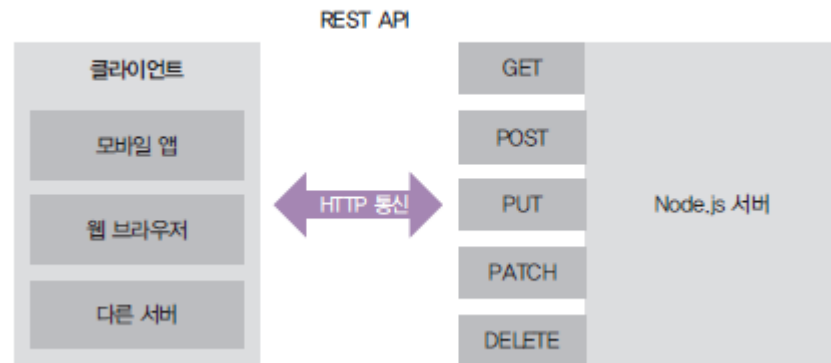
REST API

- 서버에 요청을 보낼 때는 주소를 통해 요청의 내용을 표현
 - /index.html이면 index.html을 보내 달라는 뜻
- REST (Representational State Transfer)
 - 자원을 이름(자원의 표현)으로 구분하여 해당 자원의 상태(정보)를 주고 받는 모든 것을 의미
 - HTTP URI(Uniform Resource Identifier)를 통해 자원을 명시하고, HTTP Method를 통해 해당 자원에 대한 CRUD Operation을 적용하는 것을 의미
- REST API
 - REST 기반으로 서비스 API를 구현한 것
 - /user이면 사용자 정보에 관한 정보를 요청하는 것
 - /post면 게시 글에 관련된 자원을 요청하는 것
- HTTP 요청 메서드
 - GET: 서버 자원을 가져오려고 할 때 사용
 - POST: 서버에 자원을 새로 등록하고자 할 때 사용(또는 뭘 써야할 지 애매할 때)
 - PUT: 서버의 자원을 요청에 들어있는 자원으로 치환하고자 할 때 사용
 - PATCH: 서버 자원의 일부만 수정하고자 할 때 사용
 - DELETE: 서버의 자원을 삭제하고자 할 때 사용

HTTP 프로토콜

- 클라이언트가 누구든 서버와 HTTP 프로토콜로 소통 가능
 - iOS, 안드로이드, 웹이 모두 같은 주소로 요청 보낼 수 있음
 - 서버와 클라이언트의 분리

♥ 그림 4-15 REST API



- RESTful
 - REST API를 사용한 주소 체계를 이용하는 서버
 - GET /user는 사용자를 조회하는 요청, POST /user는 사용자를 등록하는 요청

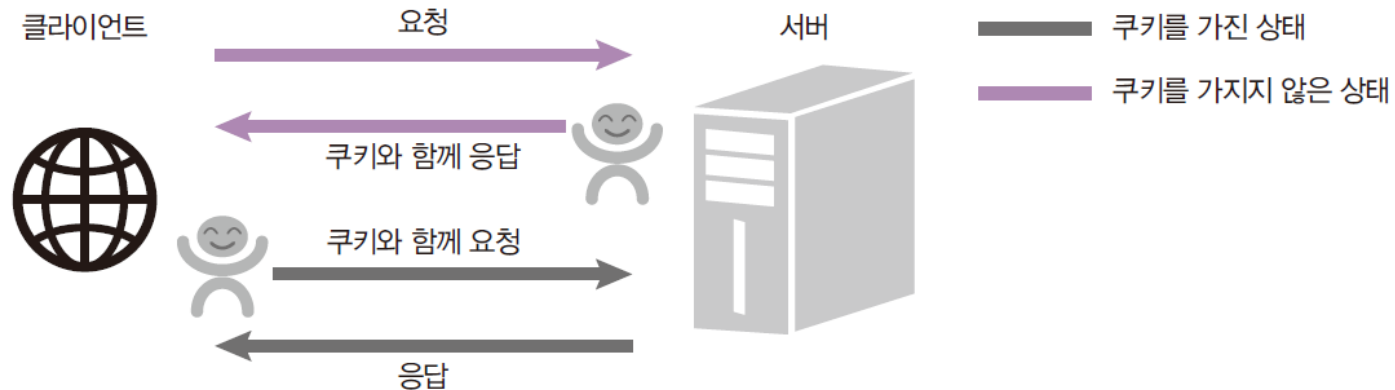
♥ 표 4-1 서버 주소 구조

HTTP 메서드	주소	역할
GET	/	restFront.html 파일 제공
GET	/about	about.html 파일 제공
GET	/users	사용자 목록 제공
GET	기타	기타 정적 파일 제공
POST	/users	사용자 등록
PUT	/users/사용자id	해당 id의 사용자 수정
DELETE	/users/사용자id	해당 id의 사용자 제거

쿠키의 필요성

- 요청에는 한 가지 단점이 있음
 - 누가 요청을 보냈는지 모름(IP 주소와 브라우저 정보 정도만 알)
 - 로그인을 구현하면 됨
 - 쿠키와 세션이 필요
- 쿠키: 키=값의 쌍
 - name=zerocho
 - 매 요청마다 서버에 동봉해서 보냄
 - 서버는 쿠키를 읽어 누구인지 파악

▼ 그림 4-13 쿠키



http 상태 코드

- writeHead 메서드에 첫 번째 인수로 넣은 값
 - 요청이 성공했는지 실패했는지를 알려줌
 - 2XX: 성공을 알리는 상태 코드입니다. 대표적으로 200(성공), 201(작성됨)이 많이 사용됩니다.
 - 3XX: 리다이렉션(다른 페이지로 이동)을 알리는 상태 코드입니다. 어떤 주소를 입력했는데 다른 주소의 페이지로 넘어갈 때 이 코드가 사용됩니다. 대표적으로 301(영구 이동), 302(임시 이동)가 있습니다.
 - 4XX: 요청 오류를 나타냅니다. 요청 자체에 오류가 있을 때 표시됩니다. 대표적으로 401(권한 없음), 403(금지됨), 404(찾을 수 없음)가 있습니다.
 - 5XX: 서버 오류를 나타냅니다. 요청은 제대로 왔지만 서버에 오류가 생겼을 때 발생합니다. 이 오류가 뜨지 않게 주의해서 프로그래밍해야 합니다. 이 오류를 클라이언트로 res.writeHead로 직접 보내는 경우는 없고, 예기치 못한 에러 발생 시 서버가 알아서 5XX대 코드를 보냅니다. 500(내부 서버 오류), 502(불량 게이트웨이), 503(서비스를 사용할 수 없음)이 자주 사용됩니다.

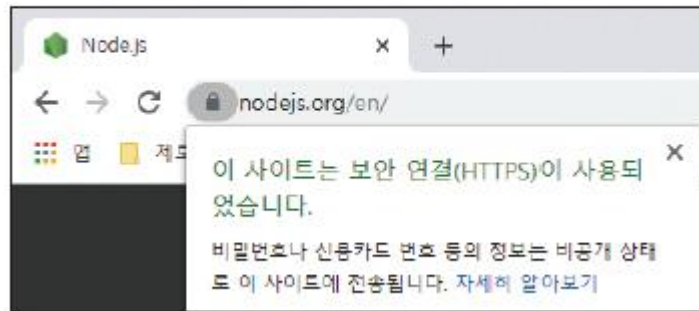
세션

- 쿠키의 정보는 노출되고 수정되는 위험이 있음
 - 중요한 정보는 서버에서 관리하고 클라이언트에는 세션 키만 제공
 - 서버에 세션 객체(session) 생성 후, 유일한 키를 만들어 속성명으로 사용
 - 속성 값에 정보 저장하고 유일한 키를 쿠키에 담아 클라이언트에 보냄

https

- 웹 서버에 SSL 암호화를 추가하는 모듈
 - 오고 가는 데이터를 암호화해서 중간에 다른 사람이 요청을 가로채더라도 내용을 확인할 수 없음
 - 요즘에는 https 적용이 필수(개인 정보가 있는 곳은 특히)

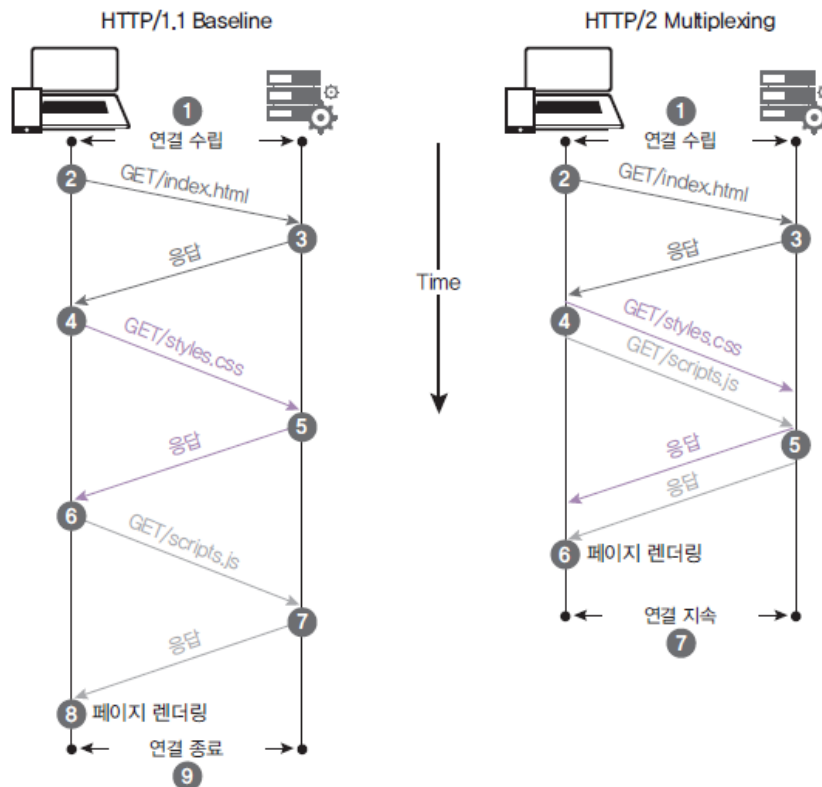
▼ 그림 4-18 https 적용 화면



http2

- SSL 암호화와 더불어 최신 HTTP 프로토콜인 http/2를 사용하는 모듈
 - 요청 및 응답 방식이 기존 http/1.1보다 개선됨
 - 웹의 속도도 개선됨

▼ 그림 4-20 http/1.1과 http/2의 비교



Cluster

- 기본적으로 싱글 스레드인 노드가 CPU 코어를 모두 사용할 수 있게 해주는 모듈
 - 포트를 공유하는 노드 프로세스를 여러 개 둘 수 있음
 - 요청이 많이 들어왔을 때 병렬로 실행된 서버의 개수만큼 요청이 분산됨
 - 서버에 무리가 덜 감
- 코어가 8개인 서버가 있을 때
 - 보통은 코어 하나만 활용
 - cluster로 코어 하나당 노드 프로세스 하나를 배정 가능
 - 성능이 8배가 되는 것은 아니지만 개선됨
- 단점
 - 컴퓨터 자원(메모리, 세션 등) 공유 못 함
 - Redis 등 별도 서버로 해결

미들웨어

- 익스프레스는 미들웨어로 구성됨
 - 요청과 응답의 중간에 위치하여 미들웨어
 - `app.use(미들웨어)`로 장착
 - 미들웨어
 - `req, res, next`가 매개변수인 함수
 - `next()`로 다음 미들웨어를 호출

▼ 표 6-1 미들웨어가 실행되는 경우

<code>app.use(미들웨어)</code>	모든 요청에서 미들웨어 실행
<code>app.use('/abc', 미들웨어)</code>	abc로 시작하는 요청에서 미들웨어 실행
<code>app.post('/abc', 미들웨어)</code>	abc로 시작하는 POST 요청에서 미들웨어 실행

app.js

```
...
app.set('port', process.env.PORT || 3000);

app.use((req, res, next) => {
  console.log('모든 요청에 다 실행됩니다.');
```

```
  next();
});

app.get('/', (req, res, next) => {
  console.log('GET / 요청에서만 실행됩니다.');
```

```
  next();
}, (req, res) => {
  throw new Error('에러는 에러 처리 미들웨어로 갑니다.');
```

```
});

app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).send(err.message);
});

app.listen(app.get('port'), () => {
  ...
});
```

module, exports

- module.exports와 exports 참조 관계
 - exports에 객체의 속성이 아닌 다른 값을 대입하면 참조 관계가 깨짐

▼ 그림 3-5 exports와 module.exports의 관계

